INF1037
Programação em C

2026.1 - Turma 33C
Prof. Adriano Branco
Aula 23: 25/05/2026

Slides da aula da Profa. Paula Rodrigues

Pontifícia Universidade Católica do Rio de Janeiro
Departamento de Informática

Conteúdo Programático

- Tipos Estruturados ✅
- Vetor de Tipos Estruturados ✅
- União e Enumeração ✅
- Arquivo Binário ✅
- Lista
- Recursão
- Segunda Prova (P2)

DEPARTAMENTO DE INFORMÁTICA
PUC·RIO

Listas Simplesmente Encadeadas
(Parte 1)

Pontifícia Universidade Católica do Rio de Janeiro
Departamento de Informática

Listas Encadeadas

- Motivação: até agora usamos estruturas de dados de tamanho fixo, como vetores unidimensionais ou bidimensionais, `structs` e `unions`. Agora vamos estudar estruturas de dados dinâmicas, que crescem e diminuem durante a execução do programa.
- A estrutura de dados dinâmica que vamos aprender no escopo desta disciplina é a **lista simplesmente encadeada** (ou apenas listas encadeadas ou listas), mas há outras como filas, pilhas, árvores binárias etc.
- Listas encadeadas são coleções de itens de dados "alinhados em uma fila" – inserções e exclusões são feitas em qualquer lugar em uma lista encadeada.

Lista encadeadas

- Listas de dados podem ser armazenadas em vetores, como vimos anteriormente, mas as listas encadeadas oferecem várias vantagens – principalmente comparadas a vetores:
  - Uma lista encadeada é apropriada quando é impossível predizer o número de elementos de dados a ser representado na estrutura de dados. Em vetores, precisamos saber a priori a quantidade de elementos.
  - As listas encadeadas são dinâmicas, assim o comprimento de uma lista pode aumentar ou diminuir de acordo com a necessidade. O tamanho de um vetor não pode ser alterado depois da memória ter sido alocada e vetores podem ficar cheios.
  - Listas encadeadas ficam cheias somente quando o sistema não tem memória suficiente para satisfazer solicitações dinâmicas de alocação de memória.

Estruturas autoreferenciadas

- Antes de entendermos a estrutura de uma lista simplesmente encadeada, vamos entender o que vem a ser uma “estrutura autorreferenciada”.

Estruturas autoreferenciadas

- Antes de entendermos a estrutura de uma lista simplesmente encadeada, vamos entender o que vem a ser uma “estrutura autorreferenciada”.
- Uma **estrutura autorreferenciada** contém um membro ponteiro que aponta para uma estrutura do mesmo tipo de estrutura.

Estruturas autoreferenciadas

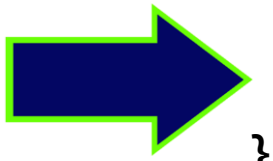
- Antes de entendermos a estrutura de uma lista simplesmente encadeada, vamos entender o que vem a ser uma "estrutura autorreferenciada".
- Uma **estrutura autorreferenciada** contém um membro ponteiro que aponta para uma estrutura do mesmo tipo de estrutura.
- Por exemplo, a definição:

```
struct no {
  tipo conteudo;
  struct no* ptr_proximo;
};
```

define um tipo, `struct no`. Essa estrutura possui dois membros: um membro que é o conteúdo propriamente dito e um membro que é um ponteiro para uma estrutura deste mesmo tipo que está sendo declarada.

Estruturas autoreferenciadas

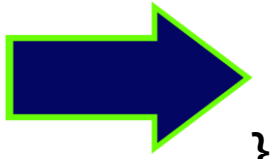
Isto é uma estrutura autorreferenciada!



```
struct no {
  tipo conteudo;
  struct no* ptr_proximo;
};
```

Estruturas autoreferenciadas

Isto é uma estrutura autorreferenciada!



```
struct no {
  tipo conteudo;
  struct no* ptr_proximo;
};
```

O membro `ptr_proximo` é chamado de ***link*** porque pode ser usado para "vincular" uma estrutura do tipo `struct no` a outra estrutura do mesmo tipo.

Quando o valor do `ptr_próximo` é `NULL`, normalmente, está indicando algo que ainda não tem elementos ou o fim de uma estrutura de dados.

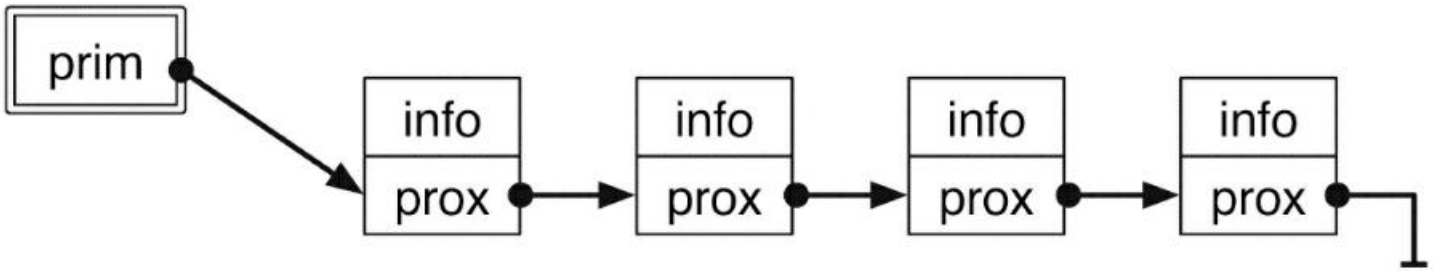
Lista encadeadas

Voltando...

- Uma lista encadeada é uma coleção linear de objetos de uma classe autorreferenciada, chamados de nós, conectados por *links* de ponteiros – daí o termo lista "encadeada".
- A lista encadeada cresce a medida que é necessário armazenar um novo elemento e ela diminui a medida que elementos são removidos.
- Uma lista encadeada é acessada por meio de um ponteiro para o **primeiro nó da lista**.
- Os nós subsequentes são acessados por meio do membro ponteiro ***link*** armazenado em cada nó.
- Por convenção, o ponteiro de ***link*** do último nó de uma lista é inicializado em **NULL**, para marcar o fim da lista.

Lista encadeadas

- Os dados são armazenados em uma lista encadeada dinamicamente – cada nó é criado de acordo com a necessidade.
- Um nó pode conter dados de qualquer tipo, inclusive outros objetos `struct`.
- Os nós de uma lista encadeada não estão armazenados consecutivamente na memória. Logicamente, porém, os nós de uma lista encadeada parecem ser contíguos.



Lista encadeadas

- Em uma lista encadeada, para cada novo elemento inserido na estrutura, alocamos um espaço de memória para armazená-lo:
  - Desta forma, o espaço total de memória gasto pela estrutura é proporcional ao número de elementos nela armazenado.
  - Como não podemos garantir que os elementos estão armazenados de forma contígua, não conseguimos ter acesso direto aos elementos da lista.
  - Para que seja possível acessar todos os elementos da lista, devemos explicitamente guardar o encadeamento dos elementos, o que é feito armazenando-se, junto com a informação, um ponteiro (ou nó) para o próximo elemento da lista.

Lista encadeadas

Com já foi visto...

- Um nó da lista é representado por uma estrutura que contém, conceitualmente, dois campos:
  - A informação armazenada; e
  - O ponteiro para o próximo elemento da lista.

➢ A partir do primeiro nó, podemos alcançar o segundo seguindo o encadeamento e assim por diante.

➢ O último nó da lista armazena, como próximo nó, um ponteiro inválido, com valor NULL, sinalizando que não existe um próximo nó.

➢ Se fôssemos desenhar um diagrama seria assim:

[Cabeça] -> [No] -> [No] -> NULL

Lista encadeadas

Observação: para simplificar, nos nossos exemplos, vamos considerar inicialmente que as informações armazenadas (o conteúdo) são números inteiros.



Lista encadeadas

- Considerando a representação de uma lista como uma estrutura autorreferenciada teremos:

```
typedef struct no_lista NoLista;
struct no_lista {
  int dado;
  NoLista *proximo;
};
```

Conteúdo da lista.

Ponteiro para o próximo elemento da lista, responsável pelo encadeamento.

Lista encadeadas

- Considerando a representação de uma lista como uma estrutura autorreferenciada teremos:

```
typedef struct no_lista NoLista;
struct no_lista {
  int dado;
  NoLista *proximo;
};
```



Considerando estas definições, podemos apresentar as principais funções necessárias para manipular listas encadeadas (usando o `int` como exemplo).

“Criando” uma lista...

```c
typedef struct no_lista NoLista;
struct no_lista {
  int dado;
  NoLista* proximo;
};

int main(void) {
  /* inicalmente nao temos uma lista criada, entao representamos com um ponteiro
  que seria a "cabeca da lista" apontando para NULL */
  NoLista *primeiro_no = NULL;
  return 0;
}
```



Inserindo um primeiro nó

```c
typedef struct no_lista NoLista;
struct no_lista {
  int dado;
  NoLista* proximo;
};

int main(void) {
  /* inicalmente nao temos uma lista criada, entao representamos com um ponteiro
  que seria a "cabeca da lista" apontando para NULL */
  NoLista *primeiro_no = NULL;
  /* criando e inserindo o primeiro nó na lista */
  NoLista *novo_no = (NoLista*)malloc(sizeof(NoLista));
  if (novo_no == NULL) {
    printf("Nao foi possivel alocar memoria para um novo no na lista.\n");
    exit(1);
  }
  printf("Forneca o valor que deseja inserir na lista: ");
  int valor;
  scanf(" %d", &valor);
  novo_no->dado = valor;
  novo_no->proximo = primeiro_no;
  primeiro_no = novo_no;
  return 0;
}
```

Inserindo um primeiro nó

```c
typedef struct no_lista NoLista;
struct no_lista {
  int dado;
  NoLista* proximo;
};

int main(void) {
  /* inicalmente nao temos uma lista criada, entao representamos com um ponteiro
  que seria a "cabeca da lista" apontando para NULL */
  NoLista *primeiro_no = NULL;
  /* criando e inserindo o primeiro nó na lista */
  NoLista *novo_no = (NoLista*)malloc(sizeof(NoLista));
  if (novo_no == NULL) {
    printf("Nao foi possivel alocar memoria para um novo no na lista.\n");
    exit(1);
  }
  printf("Forneca o valor que deseja inserir na lista: ");
  int valor;
  scanf(" %d", &valor);
  novo_no->dado = valor;
  novo_no->proximo = primeiro_no;
  primeiro_no = novo_no;
  return 0;
}
```

Vamos colocar esta parte de criação e inserção em uma função?



Inserção no início em uma função: opção 1

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct no_lista NoLista;
struct no_lista {
  int dado;
  NoLista* proximo;
};

void insere_inicio_lista(NoLista **no_lista, int valor) {
  /* criando e inserindo o primeiro nó na lista */
  NoLista* novo_no = (NoLista*) malloc(sizeof(NoLista));
  if (novo_no == NULL) {
    printf("Nao foi possivel alocar memoria para um novo no na lista.\n");
    exit(1);
  }
  novo_no->dado = valor;
  novo_no->proximo = *no_lista;
  *no_lista = novo_no;
}
...
```

Opção 1: Passar ponteiro para ponteiro
É a forma mais comum e didática.

Inserção no início em uma função: opção 1

```
...

int main (void) {
  /* inicalmente nao temos uma lista criada, entao representamos com um ponteiro
  que seria a "cabeca da lista" apontando para NULL */
  NoLista *primeiro_no = NULL;
  printf("Forneca o valor que deseja inserir na lista: ");
  int valor;
  scanf(" %d", &valor);

  insere_inicio_lista(&primeiro_no, valor);

  /* imprimindo a lista */
  printf("Imprimindo os valores da lista:\n");
  for (NoLista* p = primeiro_no; p != NULL; p = p->proximo) {
    printf("\tValor do no: %d\n", p->dado);
  }
  return 0;
}
```

Opção 1: Passa o endereço do ponteiro para que seja realmente alterado o ponteiro original.

*Na lista encadeada, a variável auxiliar (**o iterador**) tem que ser um **ponteiro** para armazenar o endereço de cada elemento. Dentro do laço da função, a variável **p** aponta para cada um dos elementos da lista, do primeiro ao último.*

Inserção no início em uma função: opção 2

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct no_lista NoLista;
  struct no_lista {
  int dado;
  NoLista* proximo;
};
```

Opção 2: Retornar o novo ponteiro
É a forma mais "limpa" para quem está começando.

```c
NoLista* insere_inicio_lista (NoLista* no_lista, int valor) {
  /* criando e inserindo o primeiro nó na lista */
  NoLista* novo_no = (NoLista*)malloc(sizeof(NoLista));
  if (novo_no == NULL) {
    printf("Nao foi possivel alocar memoria para um novo no na lista.\n");
    exit(1);
  }
  novo_no->dado = valor;
  novo_no->proximo = no_lista;
  return novo_no;
}

...
```

Inserção no início em uma função: opção 2

```
...

int main (void) {
  /* inicalmente nao temos uma lista criada, entao representamos com um ponteiro
  que seria a "cabeca da lista" apontando para NULL */
  NoLista* primeiro_no = NULL;
  printf("Forneca o valor que deseja inserir na lista: ");
  int valor;
  scanf(" %d", &valor);

  primeiro_no = insere_inicio_lista(primeiro_no, valor);

  /* imprimindo a lista */
  printf("Imprimindo os valores da lista:\n");
  for (NoLista* p = primeiro_no; p != NULL; p = p->proximo) {
    printf("\tValor do no: %d\n", p->dado);
  }
  return 0;
}
```

Opção 2: Atualiza a “cabeça” da lista.

Manipulação da lista de forma geral

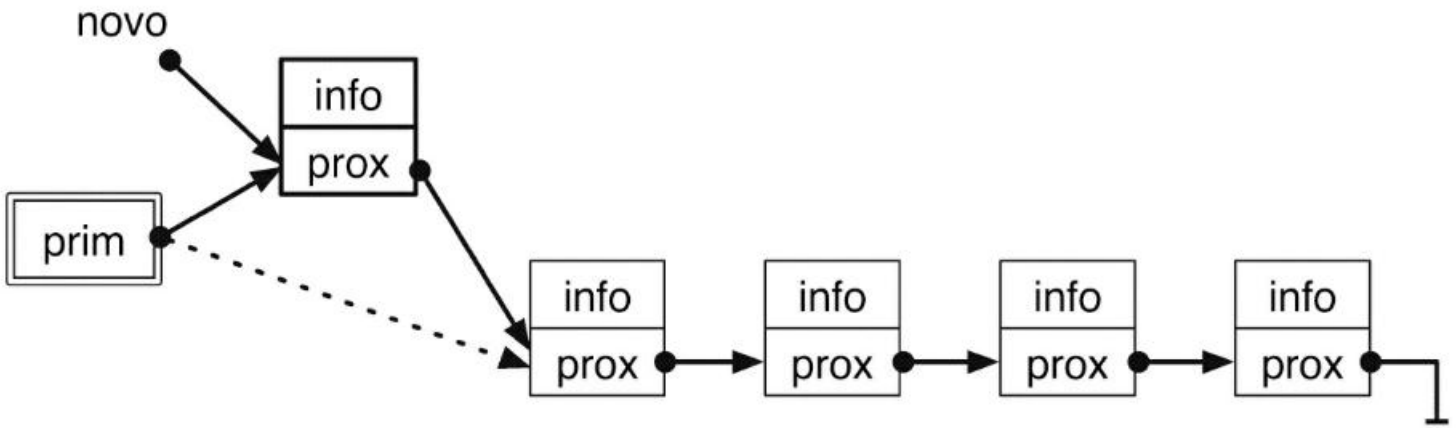
Opção 1 – PASSAGEM POR REFERÊNCIA

```
void insere_inicio_lista(NoLista **no_lista, int valor);
```

Opção 2 – PASSAGEM POR VALOR (PASSAGEM POR PONTEIRO)

```
NoLista* insere_inicio_lista(NoLista *no_lista, int valor);
```

Função de inserção no inicio em uma lista



Função que percorre os elementos da lista

```c
int main (void) {
...
  /* imprimindo a lista */
  printf("Imprimindo os valores da lista:\n");
  for (NoLista* p = primeiro_no; p != NULL; p = p->proximo) {
    printf("\tValor do no: %d\n", p->dado);
  }
..
}
```



Para simplificar, a **`imprime_lista`** *precisa receber apenas o* **`NoLista*`** *(não o ponteiro duplo), já que não altera a lista, apenas lê.*

```c
void imprime_lista(const NoLista* lista) {
  if (lista == NULL) {
    printf("A lista esta vazia.\n");
    return;
  }
  printf("Imprimindo os valores da lista:\n");
  for (const NoLista* p = lista; p != NULL; p = p->proximo) {
    printf("\tValor do no: %d\n", p->dado);
  }
}
```

```c
int main (void) {
...
  /* imprimindo a lista */
  imprime_lista(primeiro_no);
...
}
```

Crescendo nossa lista ...

```c
int main (void) {
  /* inicalmente nao temos uma lista criada, entao representamos com um ponteiro
  que seria a "cabeca da lista" apontando para NULL */
  NoLista *primeiro_no = NULL;
  printf("Forneca quantos valores deseja inserir na lista:  ");
  int qtd;
  scanf(" %d", &qtd);
  for (int i = 0; i < qtd; i++) {
    printf("Qual valor deseja inserir? ");
    int valor;
    scanf(" %d", &valor);
    insere_inicio_lista(&primeiro_no, valor);
  }
  /* imprimindo a lista */
  imprime_lista(primeiro_no);
  return 0;
}
```

```
Forneca quantos valores deseja inserir na lista:  5
Qual valor deseja inserir? 1
Qual valor deseja inserir? 2
Qual valor deseja inserir? 3
Qual valor deseja inserir? 4
Qual valor deseja inserir? 5
Imprimindo os valores da lista:
        Valor do no: 5
        Valor do no: 4
        Valor do no: 3
        Valor do no: 2
        Valor do no: 1
```

Função que verifica se determinado elemento pertence à lista

```c
/* retorna 0 se o elemento nao esta na lista e
   retorna 1 se o elemento esta na lista */
int pertence_lista(NoLista* lista, int elem) {
  if (lista == NULL) {
    printf("A lista esta vazia.\n");
    return 0;
  }
  for (NoLista* p = lista; p != NULL; p = p->proximo) {
    if (p->dado == elem) {
      return 1;
    }
  }
  return 0;
}
```

```c
int main (void) {
  ...
  /* testando pertence */
  printf("Qual elemento voce quer ver se tem na lista? ");
  int elem, achou;
  scanf(" %d", &elem);
  achou = pertence_lista(primeiro_no, elem);
  if (achou) {
    printf("O elemento %d foi encontrado na lista.\n");
  }
  else {
    printf("O elemento %d nao foi encontrado na lista.\n");
  }
  return 0;
}
```

Função que verifica se determinado elemento pertence à lista

```
Forneca quantos valores deseja inserir na lista:  5
Qual valor deseja inserir? 1
Qual valor deseja inserir? 2
Qual valor deseja inserir? 3
Qual valor deseja inserir? 4
Qual valor deseja inserir? 5
Imprimindo os valores da lista:
        Valor do no: 5
        Valor do no: 4
        Valor do no: 3
        Valor do no: 2
        Valor do no: 1
Qual elemento voce quer ver se tem na lista? 3
O elemento 3 foi encontrado na lista.
```

```
Forneca quantos valores deseja inserir na lista:  5
Qual valor deseja inserir? 1
Qual valor deseja inserir? 2
Qual valor deseja inserir? 3
Qual valor deseja inserir? 4
Qual valor deseja inserir? 5
Imprimindo os valores da lista:
        Valor do no: 5
        Valor do no: 4
        Valor do no: 3
        Valor do no: 2
        Valor do no: 1
Qual elemento voce quer ver se tem na lista? 6
O elemento 6 nao foi encontrado na lista.
```

Função que calcula o comprimento da lista

```c
int comprimento_lista(NoLista* lista) {
  int tam = 0;
  for (NoLista* p = lista; p != NULL; p = p->proximo) {
    tam++;
  }
  return tam;
}
```

```c
int main (void) {
  ...
  printf("Comprimento da lista: %d\n", comprimento_lista(primeiro_no));
  ...
  return 0;
}
```

Erros Comuns

- Esquecer `free()` -> vazamento ☹
- Acessar `p-> próximo` quando `p == NULL`
- Não atualizar a “cabeça” da lista (`primeiro`) corretamente
- Usar ponteiro sem ter inicializado

Boas Práticas

- Modularize: crie funções para cada operação
- Teste cada função isoladamente
- Verifique o retorno do `malloc`
- Documente invariantes da lista (exemplo: a cabeça)

Mundo real ☺

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct aluno Aluno;

struct aluno {
  int matricula;
  char nome[40];
  float nota;
};

typedef struct no_lista NoLista;
struct no_lista {
  Aluno conteudo;
  NoLista* proximo;
};

NoLista* cria_lista() {
  return NULL;
}
```

Mundo real ☺

```c
void insere_lista_inicio(NoLista** lista, Aluno aluno) {
  NoLista* novo_no = (NoLista*)malloc(sizeof(NoLista));
  if (novo_no == NULL) {
    printf("Nao foi possivel alocar memoria.\n");
    exit(1);
  }
  novo_no->conteudo = aluno;
  novo_no->proximo = *lista;
  *lista = novo_no;
}

void imprime_lista(const NoLista* lista) {
  if (lista == NULL) {
    printf("A lista esta vazia.\n");
    return;
  }
  printf("Lista de alunos:\n");
  for (const NoLista* p = lista; p != NULL; p = p->proximo) {
    printf("\tMatricula: %d -- Nome: %s -- Nota: %.2f\n", p->conteudo.matricula,
           p->conteudo.nome, p->conteudo.nota);
  }
}
```

Mundo real ☺

```c
void libera_lista(NoLista** lista) {
  ... /* próxima aula */
}

int main(void) {
  NoLista* lista = cria_lista();
  int qtd;
  printf("Quantos alunos deseja inserir? ");
  scanf(" %d", &qtd);
  for (int i = 0; i < qtd; i++) {
    Aluno a;
    printf("\n--- Inserindo aluno %d ---\n", i + 1);
    printf("Matricula: ");
    scanf(" %d", &a.matricula);
    printf("Nome: ");
    scanf(" %[^\n]", a.nome);
    printf("Nota: ");
    scanf(" %f", &a.nota);
    insere_lista_inicio(&lista, a);
  }
  imprime_lista(lista);
  libera_lista(&lista);
  return 0;
}
```

Nossos próximos passos...

- Inserir no final
- Inserir de forma ordenada
- Remover um elemento
- Buscar elemento por um campo
- Salvar e carregar a lista em um arquivo binário
- Ordenar a lista por um determinado critério

